

NATIONAL ECONOMICS UNIVERSITY



PROJECT

DATABASE MANAGEMENT SYSTEM

INVENTORY MANAGEMENT SYSTEM

Students: Vu Duc Anh (11245840)

Instructor: Dr. Tran Hung

Ha Noi, 2026

PROJECT 07: INVENTORY MANAGEMENT SYSTEM

DATCOM Lab
NEU-College of Technology
National Economics University
Email: hung.tran@neu.edu.vn

Project Objective

The objective of this project is to design and implement a system to efficiently manage inventory operations, track product quantities across warehouses, manage suppliers, and monitor stock movements and purchase history.

Database and Programming Languages

- **Database Management System:** MySQL
- **Programming Language:** Python

Detailed Project Requirements

System Analysis and Requirements

- Manage detailed information on products, suppliers, warehouses, stock entries, and inventory history.

Main Functionalities

- Product management (add, update, track product information and stock).
- Supplier management (store supplier contacts and information).
- Warehouse management (location, capacity, stock details).
- Manage stock entries and purchase orders.
- Track inventory history (in/out transactions per product and warehouse).
- Generate stock level alerts and inventory reports.

Database Design and Implementation

1. Data Model Design

- Design the Entity-Relationship (ER) diagram.
- Convert ER diagram to a relational schema with PKs, FKs, and constraints.

2. Table Structures

- Products (ProductID, ProductName, Description, UnitPrice, SupplierID)
- Suppliers (SupplierID, SupplierName, Address, PhoneNumber)
- Warehouses (WarehouseID, WarehouseName, Address)
- StockEntries (EntryID, ProductID, Quantity, EntryDate)
- InventoryHistory (HistoryID, ProductID, WarehouseID, Quantity, TransactionDate)

3. Sample Data

- Insert 510 records for each table.
- Generate a database schema diagram using MySQL Workbench.

Advanced Database Objects

- **Indexes:** Improve search speed for frequently queried fields (e.g., ProductName, WarehouseID).
- **Views:** Create views to summarize stock levels per warehouse or supplier delivery history.
- **Stored Procedures:** Automate product restocking, inventory calculations.
- **User Defined Functions:** Calculate stock turnover rates or average delivery time.
- **Triggers:** Update inventory levels upon stock addition or removal.

Database Security and Administration

- Manage user roles (inventory manager, admin) and set access permissions.
- Implement security measures to protect supplier and product data.
- Plan and test regular data backups and recovery operations.
- Use indexing and query optimization to improve system performance.

Python Application Development

- **Database Connection:** Use `mysql-connector-python` or `SQLAlchemy`.
- **Data Management:** Write Python functions to handle product entry, updates, and warehouse operations.
- **Querying and Reporting:** Automate the generation of low-stock alerts and inventory balance reports.
- **Interactive Interface:** Create a command-line tool or basic GUI for interacting with the inventory system.

Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Evaluate the efficiency of the implemented system.
- Suggest enhancements such as barcode scanning, integration with ERP, or predictive restocking.

References

- List all documents, tutorials, libraries, and references used during the project.

NATIONAL ECONOMICS UNIVERSITY

NEU College of Technology

FINAL PROJECT REPORT

Inventory Management System

Database Management System Final Project

Student Name:	Vũ Đức Anh
Student ID:	11245840
Class:	DSEB-66B
Course:	Database Management
Lecturer:	Hung Tran
GitHub:	github.com/tombeo0912/ inventory-management-system
YouTube Demo:	https://youtu.be/6MoBctTNVW4? si=n68q514mhyKwwDmK
Submission Date:	14/05/2026

Database Management System:	MySQL
Programming Language:	Python
Interfaces:	CLI and CustomTkinter GUI
Main Libraries:	mysql-connector-python, CustomTkinter, Matplotlib, python-dotenv

Academic Year: 2026

Hanoi, Vietnam

Contents

Abstract	1
1 Introduction	2
1.1 Project Background	2
1.2 Project Objectives	2
1.3 Project Scope	3
1.4 Technologies Used	3
2 Requirements Analysis	4
2.1 Core Data Objects	4
2.2 Functional Requirements	4
2.3 Non-functional Requirements	5
2.4 Operational Flow	6
3 Database Design	7
3.1 Entity–Relationship Diagram	7
3.2 Relational Schema Diagram	8
3.3 Relational Schema Notation	8
3.4 Design Rationale	10
3.5 Main Tables and Purposes	11
3.6 Detailed Data Dictionary	11
4 Database Implementation	17
4.1 Database Creation and Table Implementation	17
4.2 Sample Data	18

4.3	Indexes	18
4.4	Views	18
4.4.1	Stock by Warehouse	19
4.4.2	Low-Stock Products	20
4.4.3	Supplier Delivery History	20
4.5	Stored Procedures	20
4.5.1	Stock-In and Stock-Out Mechanism	21
4.6	User-Defined Functions	21
4.7	Triggers	22
4.8	Security, Roles, and Backup Strategy	22
5	Python Application Development	23
5.1	Application Architecture	23
5.2	Database Connection	23
5.3	Authentication and Role Control	24
5.4	Implemented Features	24
5.5	GUI Application	25
5.6	CLI Application	26
5.7	User Experience Improvements	26
6	System Demonstration and Testing	27
6.1	Testing Strategy	27
6.2	Test Case Summary	27
6.3	Selected GUI Screenshots	28
6.4	Observed Results	29
7	Conclusion and Future Development	30
7.1	Conclusion	30
7.2	Limitations	30
7.3	Future Improvements	31
	References	32

A Project Links	33
B Project File Structure	34

List of Figures

2.1	Overall system architecture	6
3.1	Entity–Relationship Diagram of the Inventory Management System	7
3.2	Relational schema diagram of the Inventory Management System	8
4.1	Stock movement processing mechanism	21
5.1	Dashboard of the CustomTkinter GUI application	25
5.2	Main menu of the CLI application	26
6.1	Selected GUI report screens	28
6.2	Selected GUI interface screens	29

List of Tables

1.1	Technologies used in the project	3
2.1	Core data objects	4
2.2	Functional requirements	5
3.1	Main table purposes	11
3.2	Data dictionary for suppliers	12
3.3	Data dictionary for products	13
3.4	Data dictionary for warehouses	14
3.5	Data dictionary for warehouse_stock	14
3.6	Data dictionary for stock_entries	15
3.7	Data dictionary for inventory_history	16
4.1	Implemented indexes	19
5.1	Python modules	23
6.1	Manual test case summary	27

Abstract

This report presents the design and implementation of an **Inventory Management System** developed using **MySQL** and **Python**. The system supports product management, supplier management, warehouse management, inbound and outbound stock movements, inventory history tracking, low-stock alerts, and inventory reporting.

The database design follows a relational model with primary keys, foreign keys, constraints, indexes, views, stored procedures, user-defined functions, and triggers. The application layer contains both a command-line interface and a modern graphical user interface built with CustomTkinter. This project demonstrates how database design, business rules, and application logic can work together to support real inventory operations.

Introduction

1.1 Project Background

Inventory management is a critical part of business operations. A company must know which products are available, where they are stored, how much stock is left, which suppliers provide the products, and when stock needs to be replenished. Manual management can easily lead to missing data, inconsistent stock balances, or delayed restocking.

This project implements an **Inventory Management System** that manages products, suppliers, warehouses, stock entries, inventory history, alerts, and reports. The system is designed for educational purposes but follows realistic database and application development principles.

1.2 Project Objectives

The objectives of the project are:

- design an Entity–Relationship Diagram and convert it into a relational schema;
- implement a MySQL database with constraints, relationships, and sample data;
- create advanced database objects including indexes, views, procedures, functions, and triggers;

- develop Python interfaces for user interaction;
- generate low-stock alerts and inventory reports;
- demonstrate working features through screenshots and a YouTube video.

1.3 Project Scope

The project focuses on the following modules:

- Product information and pricing;
- Supplier information and delivery history;
- Warehouse information and capacity;
- Current stock balance per product and warehouse;
- Inbound and outbound stock movement;
- Inventory transaction history;
- Stock turnover and low-stock reports;
- Role-based access for Admin and Inventory Manager.

1.4 Technologies Used

Table 1.1: Technologies used in the project

Technology	Purpose
MySQL	Database management system
Python	Application development language
mysql-connector-python	Connecting Python to MySQL
CustomTkinter	Graphical user interface
Matplotlib	Stock turnover chart visualization
dotenv	Environment variable management
Mermaid/MySQL Workbench	ERD and schema visualization

Requirements Analysis

2.1 Core Data Objects

The system manages six major data objects:

Table 2.1: Core data objects

Object	Description
Suppliers	Companies or individuals supplying products to the inventory system.
Products	Goods stored, purchased, and monitored in warehouses.
Warehouses	Storage locations with address, capacity, manager, and contact information.
Warehouse Stock	Current stock quantity for each product in each warehouse.
Stock Entries	Inbound purchase/receiving records.
Inventory History	Full IN/OUT transaction history for tracking stock movements.

2.2 Functional Requirements

Table 2.2: Functional requirements

Module	Requirement
Product Management	Add products, list products, update product price, and track reorder level.
Supplier Management	Add suppliers, list supplier information, and view delivery history.
Warehouse Management	Add warehouses, list warehouse information, and track stock by warehouse.
Stock Operations	Add inbound stock and remove outbound stock while preventing negative stock.
Inventory Tracking	Store detailed IN/OUT transaction history.
Alerts	Generate low-stock alerts when quantity is less than or equal to reorder level.
Reporting	Generate inventory balance, turnover, delivery, and full inventory reports.
Interface	Provide a CLI and GUI for interacting with the system.
Security	Support admin and manager roles with controlled feature access.

2.3 Non-functional Requirements

- **Data integrity:** use constraints, foreign keys, triggers, and stored procedures.
- **Usability:** provide clear CLI/GUI menus and user-friendly error messages.
- **Performance:** use indexes and views to support common queries.
- **Security:** separate access levels between Admin and Inventory Manager.
- **Maintainability:** split Python logic into database, services, CLI, and GUI modules.

2.4 Operational Flow

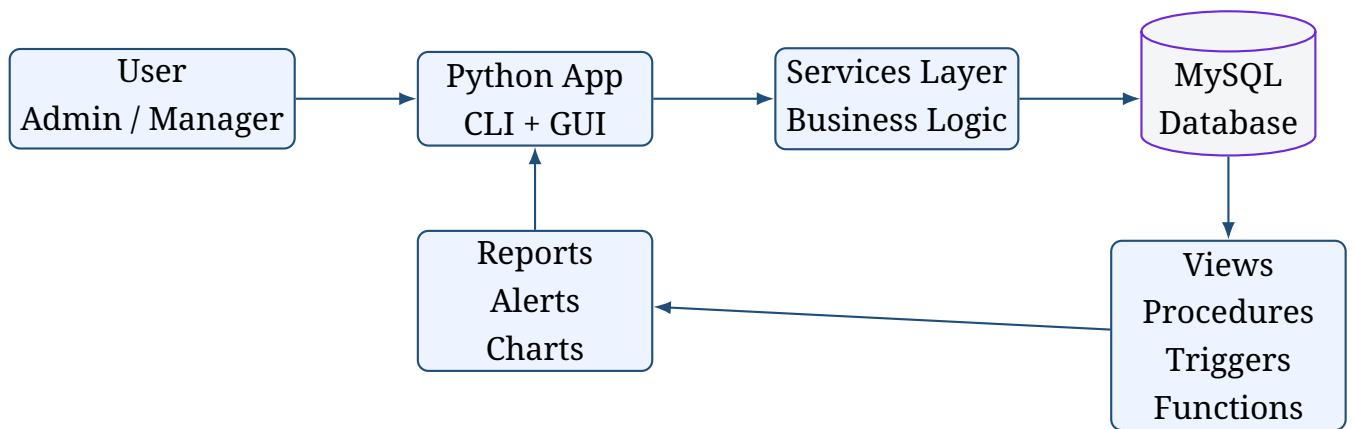


Figure 2.1: Overall system architecture

Database Design

3.1 Entity–Relationship Diagram

The ERD identifies the main entities and relationships in the inventory system. A supplier can provide many products. A product can be stored in many warehouses through the warehouse_stock table. Stock entries record inbound activities, and inventory history records both inbound and outbound transactions.

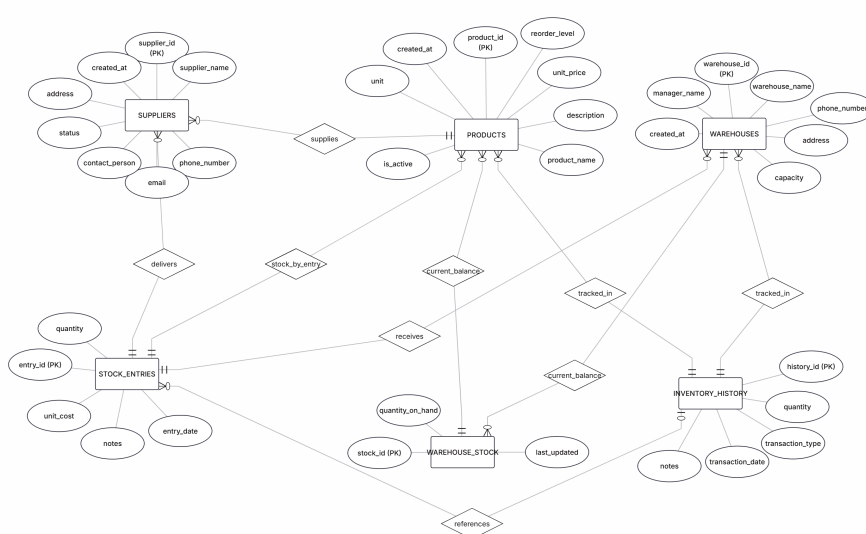


Figure 3.1: Entity–Relationship Diagram of the Inventory Management System

3.2 Relational Schema Diagram

The ERD is converted into a relational schema with primary keys, foreign keys, unique constraints, and check constraints. The relational schema diagram is used to demonstrate how the database is implemented in MySQL.

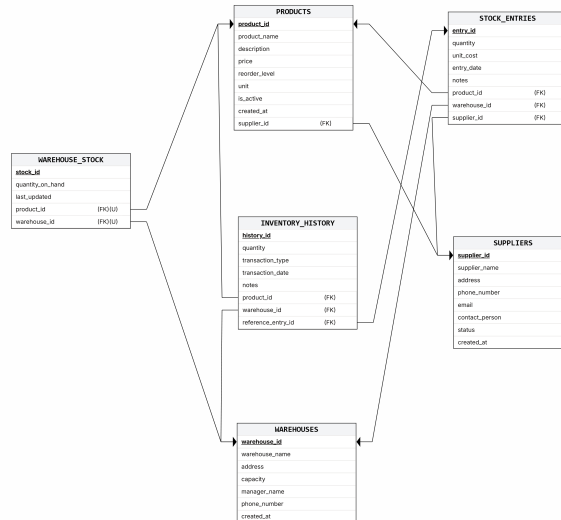


Figure 3.2: Relational schema diagram of the Inventory Management System

3.3 Relational Schema Notation

Listing 3.1: Relational schema notation

```

1  suppliers(
2      supplier_id PK,
3      supplier_name UNIQUE,
4      address,
5      phone_number,
6      email,
7      contact_person,
8      status,
9      created_at
10 )
11
12 products(

```

```
13     product_id PK,  
14     product_name,  
15     description,  
16     unit_price,  
17     supplier_id FK,  
18     reorder_level,  
19     unit,  
20     is_active,  
21     created_at  
22 )  
23  
24 warehouses(  
25     warehouse_id PK,  
26     warehouse_name UNIQUE,  
27     address,  
28     capacity,  
29     manager_name,  
30     phone_number,  
31     created_at  
32 )  
33  
34 warehouse_stock(  
35     stock_id PK,  
36     warehouse_id FK,  
37     product_id FK,  
38     quantity_on_hand,  
39     last_updated,  
40     UNIQUE(warehouse_id, product_id)  
41 )  
42  
43 stock_entries(  
44     entry_id PK,  
45     product_id FK,  
46     warehouse_id FK,  
47     supplier_id FK,  
48     quantity,  
49     unit_cost,  
50     entry_date,  
51     notes  
52 )  
53  
54 inventory_history(  
55     entry_id PK,  
56     product_id FK,  
57     warehouse_id FK,  
58     supplier_id FK,  
59     quantity,  
60     unit_cost,  
61     entry_date,  
62     notes  
63 )
```

```
55     history_id PK,  
56     product_id FK,  
57     warehouse_id FK,  
58     quantity,  
59     transaction_type,  
60     transaction_date,  
61     reference_entry_id,  
62     notes  
63 )
```

3.4 Design Rationale

The original assignment mentions products, suppliers, warehouses, stock entries, and inventory history. This implementation adds `warehouse_stock` as a practical helper table. This table stores the current balance for each product in each warehouse, which improves lookup performance and makes low-stock reports more efficient.

Without `warehouse_stock`, the application would need to repeatedly calculate current stock by summing historical transactions. With it, the system can quickly display current inventory while still keeping transaction history for traceability.

3.5 Main Tables and Purposes

Table 3.1: Main table purposes

Table	Purpose
suppliers	Stores supplier information such as name, phone, email, and contact person.
products	Stores product information such as name, description, price, supplier, reorder level, and unit.
warehouses	Stores warehouse information such as name, address, capacity, manager, and phone number.
warehouse_stock	Stores current stock quantity of each product in each warehouse.
stock_entries	Stores inbound stock entries from suppliers.
inventory_history	Stores IN and OUT transactions for audit and reporting.

3.6 Detailed Data Dictionary

Instead of presenting all fields in a single crowded table, the data dictionary is organized by database table. This layout improves readability and clearly separates the purpose of each entity.

suppliers

Field	Key / Constraint	Description
supplier_id	PK	Unique supplier identifier.
supplier_name	UNIQUE, NOT NULL	Supplier name.
address	–	Supplier address.
phone_number	CHECK	Contact phone number.
email	–	Supplier email address.
contact_person	–	Main contact person.
status	DEFAULT	Supplier active/inactive status.
created_at	DEFAULT	Record creation timestamp.

Table 3.2: Data dictionary for suppliers

products

Field	Key / Constraint	Description
product_id	PK	Unique product identifier.
product_name	NOT NULL	Product name.
description	–	Product description.
unit_price	CHECK > 0	Product unit price.
supplier_id	FK	References the supplier that provides the product.
reorder_level	CHECK ≥ 0	Minimum stock threshold for low-stock alerts.
unit	DEFAULT	Product unit such as pcs, roll, ream, or pair.
is_active	DEFAULT	Indicates whether the product is active.
created_at	DEFAULT	Record creation timestamp.

Table 3.3: Data dictionary for products

warehouses

Field	Key / Constraint	Description
warehouse_id	PK	Unique warehouse identifier.
warehouse_name	UNIQUE, NOT NULL	Warehouse name.
address	–	Warehouse address or location.
capacity	CHECK > 0	Maximum storage capacity.
manager_name	–	Name of the warehouse manager.
phone_number	–	Warehouse contact phone number.
created_at	DEFAULT	Record creation timestamp.

Table 3.4: Data dictionary for warehouses

warehouse_stock

Field	Key / Constraint	Description
stock_id	PK	Unique stock row identifier.
warehouse_id	FK	References the warehouse.
product_id	FK	References the product.
quantity_on_hand	CHECK ≥ 0	Current stock quantity of a product in a warehouse.
last_updated	DEFAULT	Last stock update timestamp.

Table 3.5: Data dictionary for warehouse_stock

stock_entries

Field	Key / Constraint	Description
entry_id	PK	Unique inbound stock entry.
product_id	FK	Product received into the warehouse.
warehouse_id	FK	Warehouse receiving the stock.
supplier_id	FK	Supplier providing the stock.
quantity	CHECK > 0	Inbound quantity.
unit_cost	CHECK > 0	Cost per unit.
entry_date	DEFAULT	Date and time of stock entry.
notes	–	Optional notes for the entry.

Table 3.6: Data dictionary for stock_entries

inventory_history

Field	Key / Constraint	Description
history_id	PK	Unique inventory history record.
product_id	FK	Product involved in the transaction.
warehouse_id	FK	Warehouse involved in the transaction.
quantity	CHECK > 0	Quantity moved in or out.
transaction_type	ENUM(IN, OUT)	Type of inventory movement.
transaction_date	DEFAULT	Date and time of the transaction.
reference_entry_id	FK, NULL	References an inbound stock entry when applicable.
notes	–	Optional transaction notes.

Table 3.7: Data dictionary for inventory_history

Database Implementation

4.1 Database Creation and Table Implementation

The database is implemented in MySQL with multiple SQL scripts. The file `01_full_project.sql` can be used to set up the entire database, while the separated scripts provide a clear breakdown of tables, indexes, functions, procedures, triggers, views, sample data, and security configuration.

Listing 4.1: Example table implementation: products

```
1 CREATE TABLE products (  
2     product_id    INT AUTO_INCREMENT PRIMARY KEY,  
3     product_name  VARCHAR(150) NOT NULL,  
4     description   VARCHAR(255),  
5     unit_price    DECIMAL(12,2) NOT NULL,  
6     supplier_id  INT NOT NULL,  
7     reorder_level INT NOT NULL DEFAULT 10,  
8     unit          VARCHAR(20) DEFAULT 'pcs',  
9     is_active     BOOLEAN DEFAULT TRUE,  
10    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,  
11  
12    CONSTRAINT chk_products_unit_price  
13        CHECK (unit_price > 0),  
14    CONSTRAINT chk_products_reorder_level  
15        CHECK (reorder_level >= 0),  
16  
17    FOREIGN KEY (supplier_id)
```

```
18 REFERENCES suppliers(supplier_id)
19 ON UPDATE CASCADE
20 ON DELETE RESTRICT
21 );
```

4.2 Sample Data

The database includes sample data for realistic testing and demonstration:

- suppliers across different business categories;
- products such as USB-C cables, keyboards, office supplies, SSDs, and monitors;
- warehouses in different locations;
- generated stock entries to support reporting and turnover calculation;
- automatically generated inventory history through triggers and procedures.

The sample data allows the application to demonstrate stock queries, warehouse reports, supplier delivery history, and low-stock alerting without requiring manual input from the user.

4.3 Indexes

Indexes are used to improve the performance of frequent searches, joins, filtering, and reporting queries. In this project, indexes are mainly created on product names, supplier references, warehouse references, stock entry dates, and inventory history fields.

4.4 Views

Views simplify reporting queries and reduce repeated SQL code in the Python application.

Table 4.1: Implemented indexes

Index	Purpose
idx_products_name	Speeds up product name searches.
idx_products_supplier	Improves joins between products and suppliers.
idx_warehouses_name	Speeds up warehouse name lookup.
idx_stock_entries_wh_date	Supports filtering stock entries by warehouse and date.
idx_stock_entries_product_date	Supports filtering stock entries by product and date.
idx_inventory_history _product_wh_date	Improves inventory history lookup by product, warehouse, and transaction date.
idx_inventory_history_type_date	Supports filtering inventory transactions by transaction type and date.
idx_warehouse_stock_qty	Supports low-stock scans and stock-level reporting.

4.4.1 Stock by Warehouse

Listing 4.2: View for current stock by warehouse

```

1 CREATE OR REPLACE VIEW vw_stock_by_warehouse AS
2 SELECT
3     ws.stock_id,
4     ws.warehouse_id,
5     w.warehouse_name,
6     ws.product_id,
7     p.product_name,
8     s.supplier_name,
9     ws.quantity_on_hand,
10    p.reorder_level,
11    p.unit,
12    CASE
13        WHEN ws.quantity_on_hand <= p.reorder_level
14        THEN 'LOW STOCK'
15        ELSE 'NORMAL'
16    END AS alert_status,
17    ws.last_updated
18 FROM warehouse_stock ws

```

```
19 JOIN warehouses w ON ws.warehouse_id = w.warehouse_id
20 JOIN products p ON ws.product_id = p.product_id
21 JOIN suppliers s ON p.supplier_id = s.supplier_id;
```

4.4.2 Low-Stock Products

Listing 4.3: Low-stock alert view

```
1 CREATE OR REPLACE VIEW vw_low_stock_products AS
2 SELECT *
3 FROM vw_stock_by_warehouse
4 WHERE quantity_on_hand <= reorder_level
5 ORDER BY quantity_on_hand ASC;
```

4.4.3 Supplier Delivery History

The supplier delivery history view aggregates inbound entries by supplier and product. This supports reporting on which suppliers delivered which products and how many units were received.

4.5 Stored Procedures

Stored procedures centralize business operations inside the database. The most important procedures are:

- `sp_add_stock`: records inbound stock and updates current balance.
- `sp_remove_stock`: validates and records outbound stock.
- `sp_inventory_report`: generates warehouse-specific inventory reports.
- `sp_generate_full_inventory_report`: generates a full inventory report.

4.5.1 Stock-In and Stock-Out Mechanism

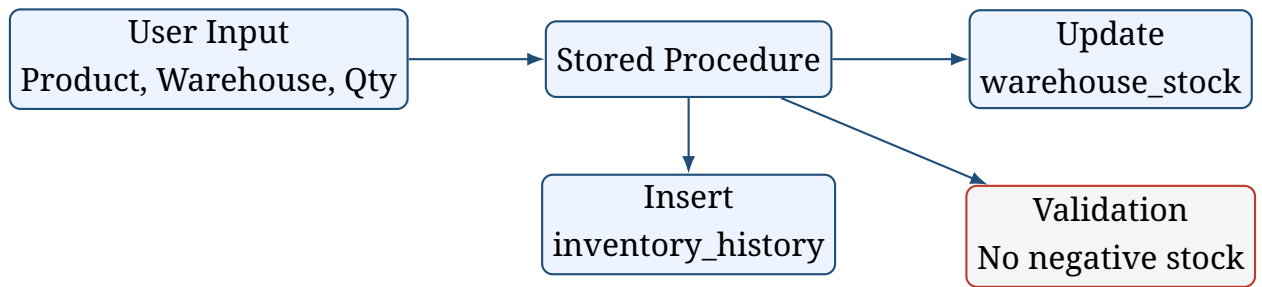


Figure 4.1: Stock movement processing mechanism

4.6 User-Defined Functions

The system uses user-defined functions to encapsulate frequently used calculations.

Listing 4.4: Function for current stock

```

1 CREATE FUNCTION fn_current_stock(
2     p_product_id INT,
3     p_warehouse_id INT
4 )
5 RETURNS INT
6 DETERMINISTIC
7 READS SQL DATA
8 BEGIN
9     DECLARE v_qty INT DEFAULT 0;
10
11     SELECT COALESCE(quantity_on_hand, 0)
12     INTO v_qty
13     FROM warehouse_stock
14     WHERE product_id = p_product_id
15           AND warehouse_id = p_warehouse_id;
16
17     RETURN COALESCE(v_qty, 0);
18 END;
```

The second function, `fn_stock_turnover`, calculates an approximate stock turnover rate based on outbound transaction quantity and average stock level. It is used in the turnover report and chart.

4.7 Triggers

Triggers are used to automatically update transaction history when stock data changes. For example, when an inbound stock entry is inserted, the system automatically creates a corresponding inventory history row. This ensures traceability and reduces the risk of missing history records.

4.8 Security, Roles, and Backup Strategy

The project implements database and application-level security concepts:

- **Admin:** full access to database and application functions.
- **Inventory Manager:** restricted operational access.
- **Backups:** the project includes backup notes using `mysqldump`.
- **Recovery:** restoration can be performed by importing backup SQL files.

A practical backup command is:

Listing 4.5: Backup command example

```
1 mysqldump -u root -p personal_finance > inventory_backup.sql
```

Python Application Development

5.1 Application Architecture

The application layer is written in Python and divided into modules. The GUI and CLI do not directly contain SQL logic; instead, they call functions in `services.py`. This separation improves maintainability.

Table 5.1: Python modules

Module	Purpose
db.py	Reads database configuration and returns MySQL connections.
services.py	Contains business logic and database operations.
main.py	Implements the CLI application.
GUI.py	Implements the CustomTkinter GUI application.
requirements.txt	Lists required Python packages.

5.2 Database Connection

The application connects to MySQL through a reusable helper function.

Listing 5.1: MySQL connection pattern

```
1 import os
2 from dotenv import load_dotenv
```

```
3 import mysql.connector
4
5 load_dotenv()
6
7 DB_CONFIG = {
8     "host": os.getenv("MYSQL_HOST", "localhost"),
9     "port": int(os.getenv("MYSQL_PORT", "3306")),
10    "user": os.getenv("MYSQL_USER", "root"),
11    "password": os.getenv("MYSQL_PASSWORD", ""),
12    "database": os.getenv("MYSQL_DATABASE", "personal_finance"),
13 }
14
15 def get_connection():
16     return mysql.connector.connect(**DB_CONFIG)
```

5.3 Authentication and Role Control

The application supports two demonstration roles:

- **Admin:** can use all features.
- **Inventory Manager:** can access view/report features and selected operational functions.

This role separation demonstrates basic application-level access control and supports the security requirement of the project.

5.4 Implemented Features

The application implements 17 major features:

1. List products
2. Add product
3. Update product price
4. List suppliers
5. Add supplier

6. List warehouses
7. Add warehouse
8. Add stock (inbound)
9. Remove stock (outbound)
10. View stock by warehouse
11. Low-stock alerts
12. Supplier delivery history
13. Inventory history
14. Check current stock
15. Stock turnover report
16. Full inventory report
17. Turnover chart visualization

5.5 GUI Application

The GUI was built using CustomTkinter. It provides a modern dashboard, navigation sidebar, data tables, forms, and reporting pages. The GUI is the main presentation interface for the final demo.

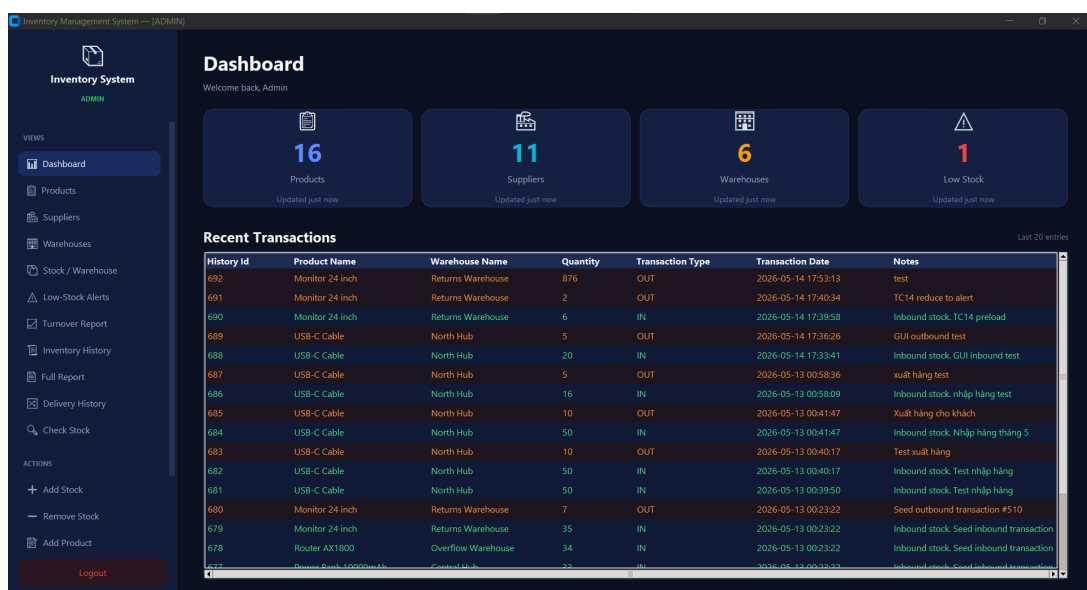


Figure 5.1: Dashboard of the CustomTkinter GUI application.

5.6 CLI Application

The CLI version is kept as a functional alternative. It provides the same core operations through terminal menus and is useful for quick testing.

```
=====
      📦 INVENTORY MANAGEMENT SYSTEM 📦 [ADMIN]
=====
1. List products
2. Add product
3. Update product price
4. List suppliers
5. Add supplier
6. List warehouses
7. Add warehouse
8. Add stock (inbound)
9. Remove stock (outbound)
10. View stock by warehouse
11. Low-stock alerts
12. Supplier delivery history
13. Inventory history
14. Check current stock for one product in one warehouse
15. Stock turnover report
16. Full inventory report (sp_generate_inventory_report)
17. Turnover chart (matplotlib, top 5)
0. Logout / Exit
=====

Choose an option: 
```

Figure 5.2: Main menu of the CLI application

5.7 User Experience Improvements

The final GUI includes several user experience improvements:

- clear navigation between views and actions;
- dashboard cards for quick overview;
- tables for products, suppliers, warehouses, stock, and reports;
- user-friendly pop-up messages;
- error handling for invalid supplier IDs, invalid prices, insufficient stock, and invalid quantities.

System Demonstration and Testing

6.1 Testing Strategy

Testing was performed manually using the GUI and CLI. The goal was to verify that the application can correctly insert, update, query, and report inventory data, while respecting database constraints.

6.2 Test Case Summary

Table 6.1: Manual test case summary

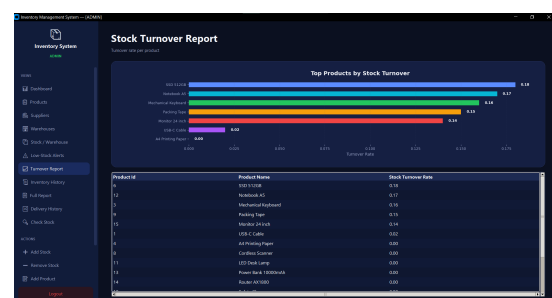
Test ID	Scenario	Expected Result	Status
TC01	Login as Admin	Dashboard opens with full access	Pass
TC02	Login as Inventory Manager	Dashboard opens with restricted access	Pass
TC03	Wrong password	Login is rejected	Pass
TC07	Add product with valid data	Product is inserted and appears in Products view	Pass
TC08	Update product price	New price appears correctly	Pass

TC09	Add supplier	Supplier appears in Suppliers view	Pass
TC10	Add warehouse	Warehouse appears in Pass Warehouses view	Pass
TC11	Add stock inbound	Stock quantity increases and history contains IN transaction	Pass
TC12	Remove stock outbound	Stock quantity decreases and history contains OUT transaction	Pass
TC13	Remove stock above available quantity	System rejects the operation	Pass
TC14	Trigger low-stock alert	Product appears in Low-Stock Alerts view	Pass
TC15	Check current stock	Result matches Stock by Warehouse view	Pass
TC16	Delivery history	Delivery history view opens correctly	Pass
TC17	Full report	Full inventory report opens correctly	Pass
TC18	Turnover report	Turnover table/chart is displayed	Pass

6.3 Selected GUI Screenshots

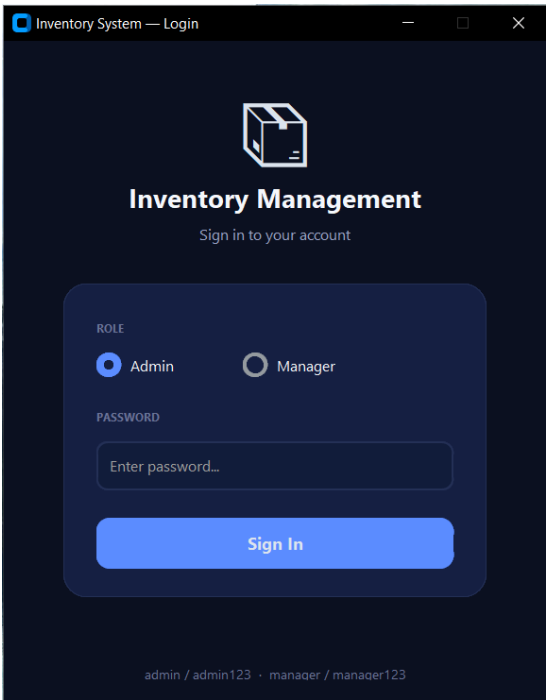


(a) Stock by Warehouse

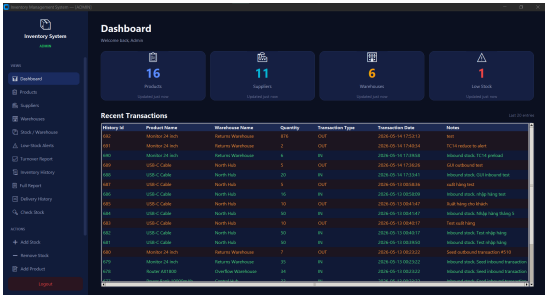


(b) Stock Turnover Report

Figure 6.1: Selected GUI report screens



(a) Login Screen



(b) Dashboard

Figure 6.2: Selected GUI interface screens

6.4 Observed Results

The implemented system successfully performs Transactions stock operations, updates current balances, records history, and generates reports. The low-stock alert is triggered correctly when stock is less than or equal to the reorder level. The outbound stock procedure also prevents negative stock by rejecting excessive outbound quantities.

The GUI version provides a more presentable interface for the final demonstration, while the CLI remains useful for quick feature testing.

Conclusion and Future Development

7.1 Conclusion

This project successfully designed and implemented an **Inventory Management System** using MySQL and Python. The system satisfies the main project requirements by managing products, suppliers, warehouses, stock entries, inventory history, low-stock alerts, and inventory reports.

The project also demonstrates advanced database concepts including indexes, views, stored procedures, user-defined functions, triggers, role-based access, and backup planning. On the application side, both CLI and GUI interfaces were developed to interact with the database and demonstrate real system functionality.

7.2 Limitations

Although the system is functional, it still has some limitations:

- The application is designed for local demonstration rather than deployment on a production server.
- The GUI is suitable for project demonstration but could be improved with more advanced styling and responsive layouts.

- Supplier-product consistency can be strengthened with additional validation rules.
- Warehouse capacity is stored but not fully enforced in every stock operation.

7.3 Future Improvements

Future enhancements may include:

- barcode scanning for faster stock operations;
- web-based dashboard for remote access;
- predictive restocking using historical demand;
- ERP or procurement system integration;
- PDF/Excel export for inventory reports;
- more advanced authentication and audit logging.

References

1. MySQL Documentation. <https://dev.mysql.com/doc/>
2. Python Documentation. <https://docs.python.org/3/>
3. MySQL Connector/Python Documentation. <https://dev.mysql.com/doc/connector-python/en/>
4. CustomTkinter Documentation. <https://customtkinter.tomschimansky.com/>
5. Matplotlib Documentation. <https://matplotlib.org/stable/>
6. Course materials and project assignment provided by NEU College of Technology.



Project Links

- GitHub Repository: github.com/tombeo0912/inventory-management-system
- YouTube Demo Video: <https://youtu.be/6MoBctTNVW4?si=n68q514mhyKwwDmK>

Project File Structure

Listing B.1: Project folder structure

```
1 inventory_final_project/
2 |
3 |-- assets/
4 |   |-- original_assignment.pdf
5 |
6 |-- docs/
7 |   |-- screenshots/
8 |       |-- gui_chart.png
9 |       |-- gui_dashboard.png
10 |       |-- gui_login.png
11 |       |-- gui_stock.png
12 |   |-- cli_preview.png
13 |   |-- ERDiagram of Inventory Management System.png
14 |   |-- inventory_erd.mmd
15 |   |-- Relational Schema Diagram of the Inventory Management
    System.png
16 |   |-- relational_schema.md
17 |
18 |-- python_app/
19 |   |-- .env.example
20 |   |-- db.py
21 |   |-- GUI.py
22 |   |-- main.py
23 |   |-- requirements.txt
```

```
24 |  `-- services.py
25 |
26 |  -- sql/
27 |    |-- 01_full_project.sql
28 |    |-- 01_create_database_and_tables.sql
29 |    |-- 02_indexes.sql
30 |    |-- 03_functions_triggers_procedures.sql
31 |    |-- 04_views.sql
32 |    |-- 05_sample_data.sql
33 |    |-- 06_security_backup.sql
34 |    `-- 07_demo_queries.sql
35 |
36 |  -- submission/
37 |    `-- final_submission_report.pdf
38 |
39 |  `-- README.md
```